

所要具备的一些基础

1. C语言
2. STM32

内容

1. 开发板各个模块的使用
2. 个人的一些编程的思想和套路

方式

把原理搞清楚
把代码敲出来
把效果演示出来

完全免费！！



[【备战2024蓝桥杯 嵌入式组】
CT117E-M4 新款开发板 3小时省赛模
块 速成总结 哔哩哔哩 bilibili](#)



[【小蜜蜂笔记】基于STM32CubeMX的嵌入
式开发基础教程 哔哩哔哩 bilibili](#)

[CSDN - 专业开发者社区](#)

蓝桥杯嵌入式省赛 - 零基础速通教程

基础篇

进阶篇

第一阶段

1. 点亮一个LED
2. 利用按键控制LED点亮和熄灭
3. LCD屏幕的简单显示
4. 利用定时器实现LED闪烁
(定时器中断使能。计数器使能)

1. LED点亮熄灭函数封装
2. 按键扫描函数封装+长按键短按键区分
3. LCD高亮显示
4. 解决LCD与LED引脚冲突问题

第二阶段

1. 定时器PWM输出 (PWM使能)
2. 定时器输入捕获

第三阶段

1. 串口接收与发送
2. adc测量

1. 串口接收复杂数据处理的三种方式

第四阶段

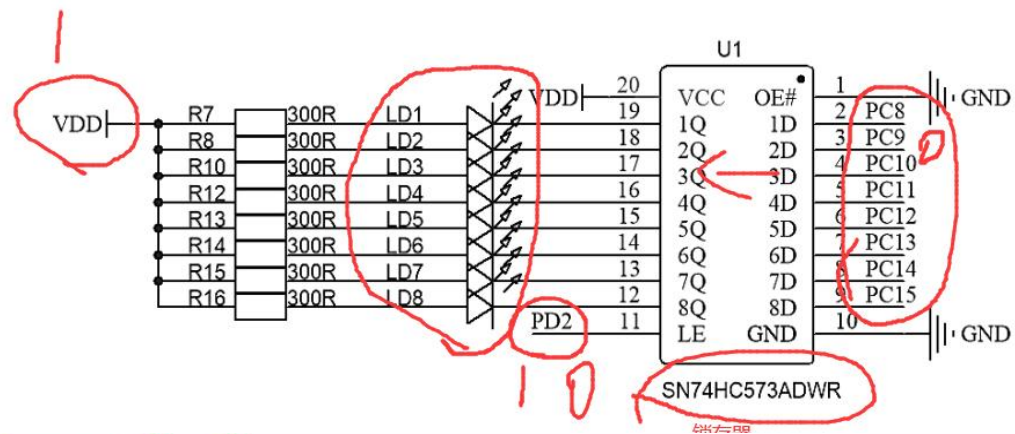
1. eeprom存储和读取
2. rtc时钟

stm32开发方式

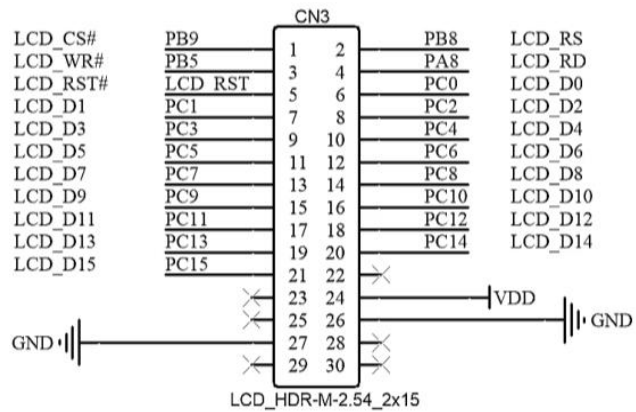
1. 基本库开发
2. HAL库开发(利用STM32CubeMX)
3. LL库开发
4. 寄存器开发

7.4 LED 指示灯

step 1 : PD2输出高电平
step 2 : PC8-PC15输出低电平
step 3 : PD2输出低电平



7.6 LCD



```
void LCD_Init(void)
```

```
{
    uint16_t temp = GPIOC->ODR;
    LCD_CtrlLinesConfig();
    dummy = LCD_ReadReg(0);

    if(dummy == 0x8230)
    {
        REG_8230_Init();
    }
    else
    {
        REG_932X_Init();
    }
    dummy = LCD_ReadReg(0);
    GPIOC->ODR = temp;
}
```

ODR: 控制PC8-PC15输出, 存的输出值

先把PC8-PC15输出值 复制给temp，
最后再复制给 ODR寄存器。解决LCD
初始化改变PC8-PC15值的问题，进而
影响LED

```
2      /* USER CODE BEGIN WHILE */  
3      while (1)  
4      {
```

所有代码一定要写在BEGIN 和 END 之间

```
6      /* USER CODE END WHILE */  
7
```

```

if(mode)
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_8 << (led - 1), GPIO_PIN_RESET);
else

```

<< 左移运算符
 0100 << 1 = 1000

```

2 #define GPIO_PIN_7      ((uint16_t)0x0080) /* Pin 7 selected */
3 #define GPIO_PIN_8      ((uint16_t)0x0100) /* Pin 8 selected */ 0000 0001 0000 0000
4 #define GPIO_PIN_9      ((uint16_t)0x0200) /* Pin 9 selected */ 0000 0010 0000 0000
5 #define GPIO_PIN_10     ((uint16_t)0x0400) /* Pin 10 selected */ 0000 0100 0000 0000
6 #define GPIO_PIN_11     ((uint16_t)0x0800) /* Pin 11 selected */
7 #define GPIO_PIN_12     ((uint16_t)0x1000) /* Pin 12 selected */
8 #define GPIO_PIN_13     ((uint16_t)0x2000) /* Pin 13 selected */
9 #define GPIO_PIN_14     ((uint16_t)0x4000) /* Pin 14 selected */
0 #define GPIO_PIN_15     ((uint16_t)0x8000) /* Pin 15 selected */
1 #define GPIO_PIN_All    ((uint16_t)0xFFFF) /* All pins selected */

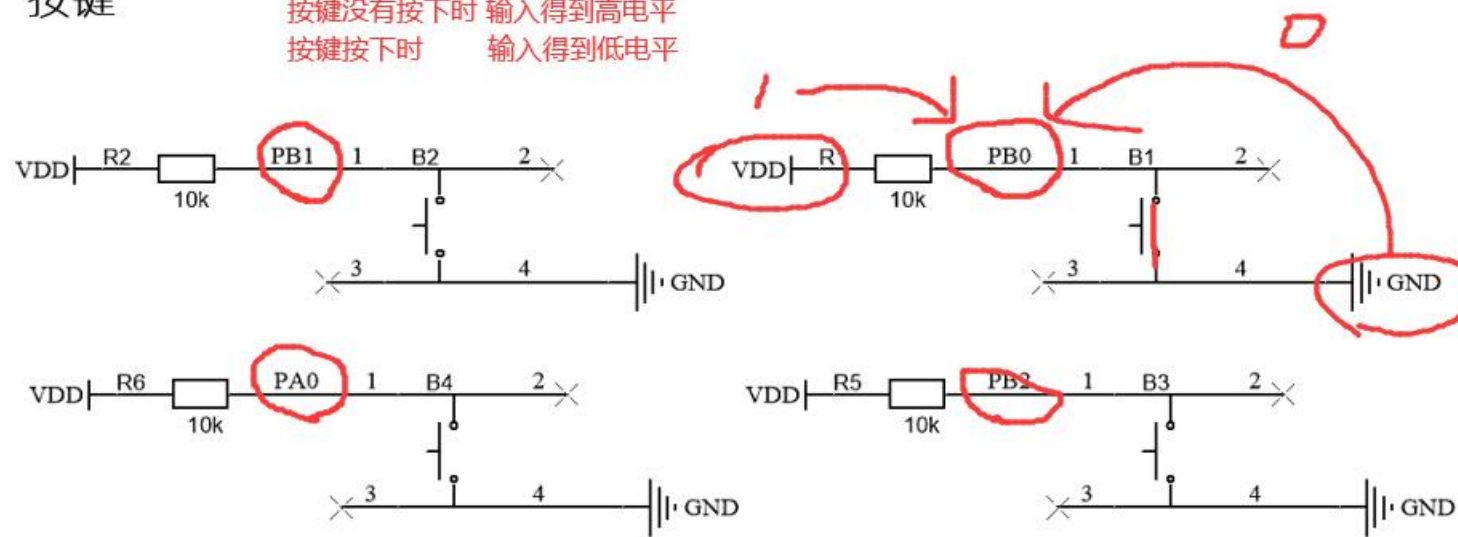
```

7.7 按键

上拉输入

按键没有按下时 输入得到高电平

按键按下时 输入得到低电平



定时器中断

```
while(1)  
{
```

.....

每隔T秒，跳出循环

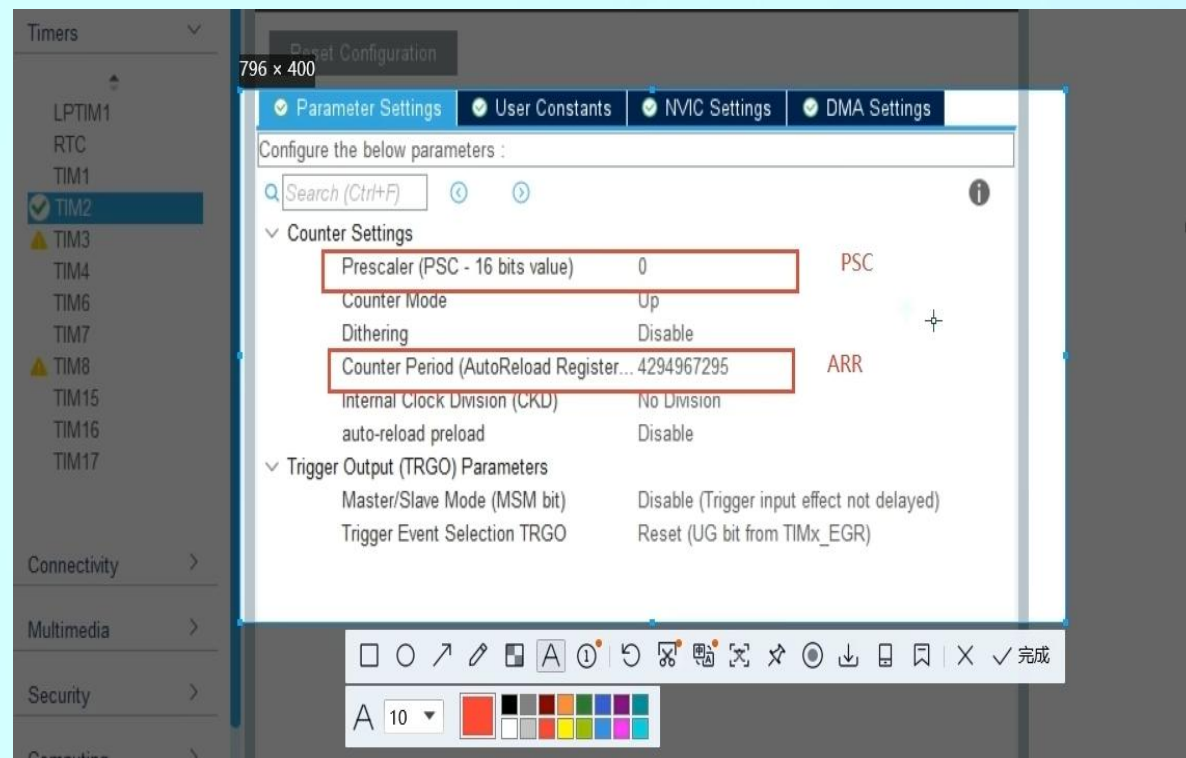
```
}
```

执行完成中断函数之后，再次回到循环

执行中断函数

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)  
{  
    if(htim->Instance == TIM2) //TIM2中断函数  
    {  
        led_mode++;  
        led_mode = led_mode % 2;  
    }  
}
```

T秒由 PSC ARR 这两个参数 计算出来



定时器时钟频率: $f = \frac{f_{system}}{(ARR + 1)(PSC + 1)}$

ARR : 自动重装载值

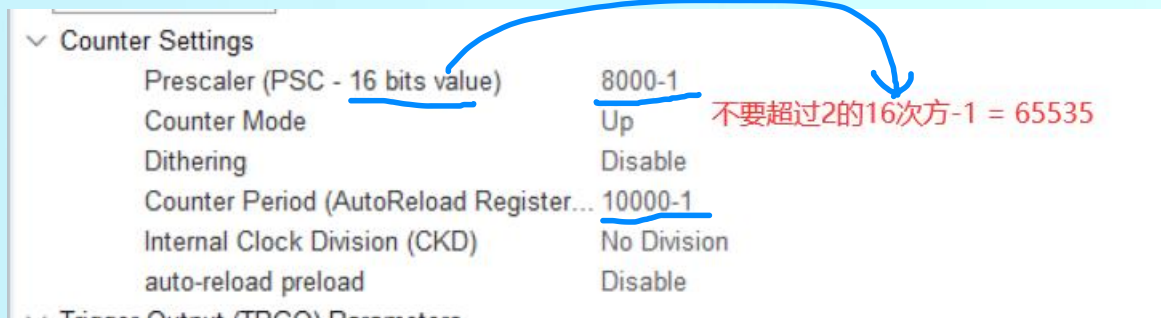
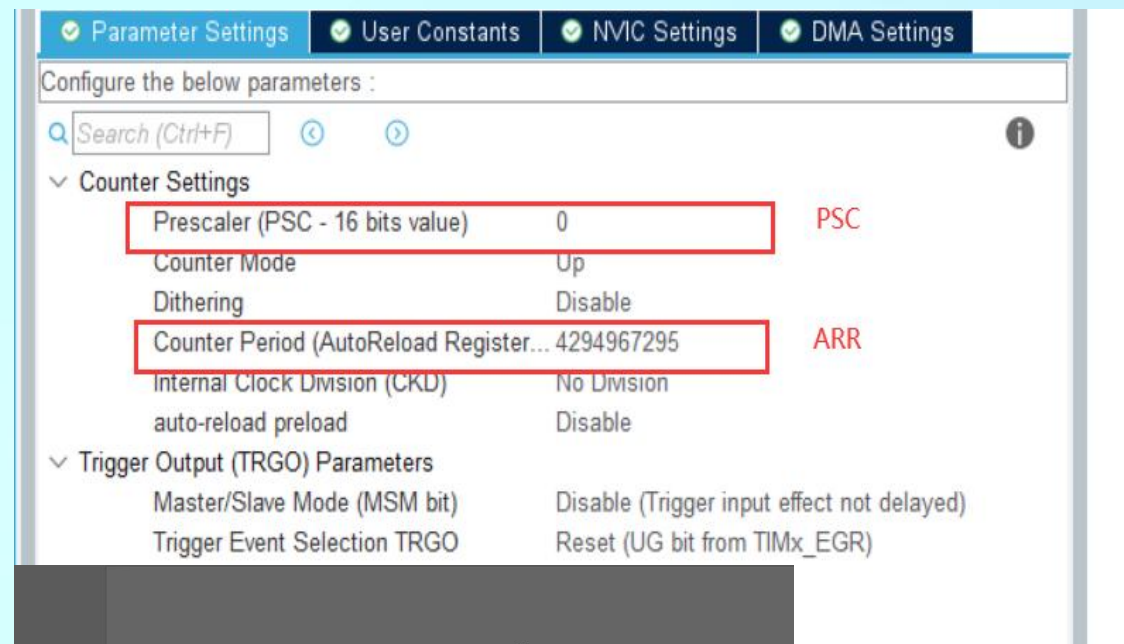
PSC: 预分频器

CNT: 计数器

定时器中断周期: $T = \frac{1}{f} = \frac{(ARR + 1)(PSC + 1)}{f_{system}}$

系统频率 80MHZ= 80 000000HZ

每隔 T 秒, 进入一次定时器中断



定时器时钟频率: $f = \frac{f_{system}}{(ARR + 1)(PSC + 1)}$

定时器中断周期: $T = \frac{1}{f} = \frac{(ARR + 1)(PSC + 1)}{f_{system}}$

80MHZ

STM32CubeMX

FileWindowHelp

HomeSTM32G431RBTxtest01_goio_lcd.ioc - Pinout & Configuration

Pinout & Configuration

Clock Configuration

Software PacksPinout

Search

CategoriesA-Z

DMA

✓ GPIO

IWDG

✓ NVIC

▲ RCC

▲ SYS

WWDG

Analog>

Timers>

LPTIM1

RTC

TIM1

✓ TIM2

▲ TIM3

TIM4

TIM6

TIM7

▲ TIM8

TIM15

TIM16

TIM17

Connectivity>

Multimedia>

Security>

Computing>

Middleware and Software Packs>

Utilities>

TIM2 Mode and Configuration

Mode

Slave ModeDisable

Trigger SourceDisable

Clock SourceInternal Clock

Channel1Disable

Channel2Disable

Channel3Disable

Channel4Disable

Combined ChannelsDisable

Use ETR as Clearing SourceDisable

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

DMA Settings

Configure the below parameters :

Search (Ctrl+F)

⏪⏩

i

Counter Settings

Prescaler (PSC - 16 bits value)8000-1

Counter ModeUp

DitheringDisable

Counter Period (AutoReload Register ...)10000-1

Internal Clock Division (CKD)No Division

auto-reload preloadDisable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit)Disable (Trigger input effect not delayed)

Trigger Event Selection TRGOReset (UG bit from TIMx_EGR)

```

58     B3_last_state = B3_state;
59     B4_last_state = B4_state;
60 }
61 }
62
63 char text[20];
64 void lcd_show()
65 {
66     sprintf(text, "      test      "); //test字符串拷贝到 text数组里
67     LCD_DisplayStringLine(Line0, (uint8_t *)text); //也会改变PC8-PC15的引脚输出
68     sprintf(text, "      count: %d      ", count); //test字符串拷贝到 text数组里
69     LCD_DisplayStringLine(Line2, (uint8_t *)text);
70 }
71 }
72
73 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
74 {
75     if(htim->Instance == TIM2) //TIM2中断函数
76     {
77         count++;
78     }
79 }

```

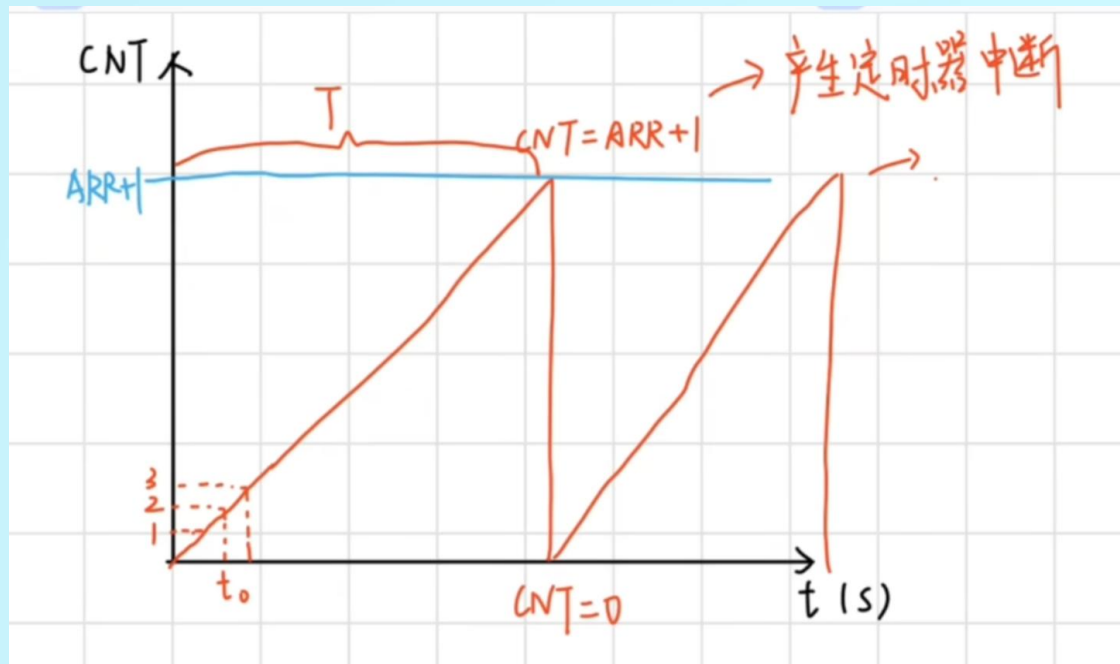
定时器中断函数

```

2358 HAL_StatusTypeDef HAL_TIM_Base_DeInit(TIM_HandleTypeDef *htim);
2359 void HAL_TIM_Base_MspInit(TIM_HandleTypeDef *htim);
2360 void HAL_TIM_Base_MspDeInit(TIM_HandleTypeDef *htim);
2361 /* Blocking mode: Polling */
2362 HAL_StatusTypeDef HAL_TIM_Base_Start(TIM_HandleTypeDef *htim);
2363 HAL_StatusTypeDef HAL_TIM_Base_Stop(TIM_HandleTypeDef *htim);
2364 /* Non-Blocking mode: Interrupt */
2365 HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim);
2366 HAL_StatusTypeDef HAL_TIM_Base_Stop_IT(TIM_HandleTypeDef *htim);
2367 /* Non-Blocking mode: DMA */
2368 HAL_StatusTypeDef HAL_TIM_Base_Start_DMA(TIM_HandleTypeDef *htim, uint32_t *pData, uint16_t Length);
2369 HAL_StatusTypeDef HAL_TIM_Base_Stop_DMA(TIM_HandleTypeDef *htim);
2370 /**

```

定时器中断使能函数



$$t_0 = \frac{1}{f_0} = \frac{1}{\frac{f_{sys}}{PSC+1}} = \frac{PSC+1}{f_{sys}}$$

$$T = t_0 \cdot (ARR+1) = \frac{(PSC+1)(ARR+1)}{f_{sys}}$$

在某个时刻 $CNT = 0$
 过一段时间 $CNT = 10000$

$$\left. \begin{array}{l} \text{在某个时刻 } CNT = 0 \\ \text{过一段时间 } CNT = 10000 \end{array} \right\} t = 10000 \cdot t_0 = 10000 \cdot \frac{PSC+1}{f_{sys}} = 1s$$

按键按下
 在某个时刻 $CNT = 0$
 过一段时间 $CNT = 10000$

$$\left. \begin{array}{l} \text{在某个时刻 } CNT = 0 \\ \text{过一段时间 } CNT = 10000 \end{array} \right\} t = 10000 \cdot t_0 = 10000 \cdot \frac{PSC+1}{f_{sys}} = 1s$$

if (CNT ≥ 10000) // 按键长按

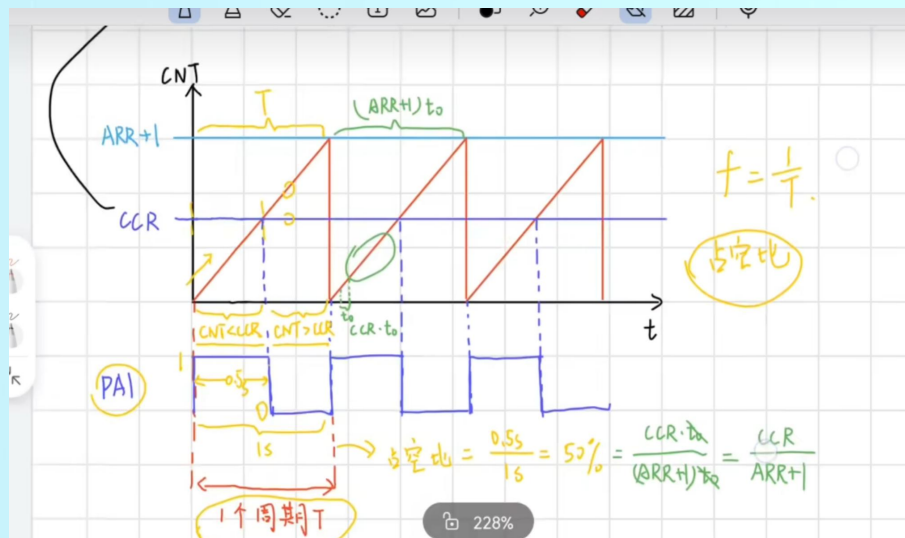
让PA1输出频率为1000HZ，占空比为50%的方波

频率:
$$f = \frac{f_{system}}{(ARR + 1)(PSC + 1)}$$

周期:
$$T = \frac{1}{f} = \frac{(ARR + 1)(PSC + 1)}{f_{system}}$$

占空比 =
$$\frac{CCR}{ARR + 1}$$

CCR - 输出比较寄存器



让PA1输出频率为1000HZ，占空比为50%的方波

660 x 156

80000000

频率:
$$f = \frac{f_{system}}{(ARR + 1)(PSC + 1)}$$

100 800

CCR

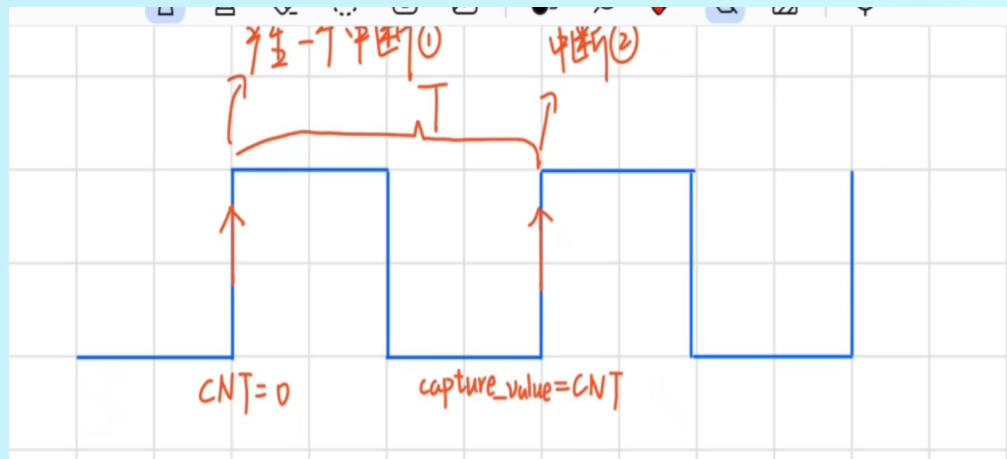
周期:
$$T = \frac{1}{f} = \frac{(ARR + 1)(PSC + 1)}{f_{system}}$$

492 x 124

50

占空比 =
$$\frac{CCR}{ARR + 1}$$

100



$$CNT=0 \quad capture_value=CNT$$

$$t_o = \frac{1}{f_{sys}} = \frac{PSC+1}{f_{sys}}$$

$$T = t_o \cdot capture_value$$

$$\star f = \frac{1}{T} = \frac{f_{sys}}{(PSC+1) \cdot capture_value}$$

```

1421 * @brief Starts the PWM signal generation.
1422 * @param htim TIM handle
1423 * @param Channel TIM Channels to be enabled
1424 * This parameter can be one of the following values:
1425 * @arg TIM_CHANNEL_1: TIM Channel 1 selected
1426 * @arg TIM_CHANNEL_2: TIM Channel 2 selected
1427 * @arg TIM_CHANNEL_3: TIM Channel 3 selected
1428 * @arg TIM_CHANNEL_4: TIM Channel 4 selected
1429 * @arg TIM_CHANNEL_5: TIM Channel 5 selected
1430 * @arg TIM_CHANNEL_6: TIM Channel 6 selected
1431 * @retval HAL status
1432 */
1433 HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)
1434 {
1435     uint32_t tmpsmcr;
1436

```

PWM使能函数

```

2523 * @}
2524 */
2525
2526 /** @defgroup TIM_Exported_Functions_Group9 TIM Callbacks functions
2527 * @brief TIM Callbacks functions
2528 * @{
2529 */
2530 /* Callback in non blocking modes (Interrupt and DMA) */
2531 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim);
2532 void HAL_TIM_PeriodElapsedHalfCpltCallback(TIM_HandleTypeDef *htim);
2533 void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim);
2534 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim);
2535 void HAL_TIM_IC_CaptureHalfCpltCallback(TIM_HandleTypeDef *htim);
2536 void HAL_TIM_PWM_PulseFinishedCallback(TIM_HandleTypeDef *htim);
2537 void HAL_TIM_PWM_PulseFinishedHalfCpltCallback(TIM_HandleTypeDef *htim);
2538 void HAL_TIM_TriggerCallback(TIM_HandleTypeDef *htim);
2539 void HAL_TIM_TriggerHalfCpltCallback(TIM_HandleTypeDef *htim);
2540 void HAL_TIM_ErrorCallback(TIM_HandleTypeDef *htim);
2541
2542 /* Callbacks Register/UnRegister functions */
2543 #if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
2544 HAL_StatusTypeDef HAL_TIM_RegisterCallback(TIM_HandleTypeDef *htim, HAL_TIM_CallbackIDTypeDef CallbackID,
2545 pTIM_CallbackTypeDef pCallback);
2546 HAL_StatusTypeDef HAL_TIM_UnRegisterCallback(TIM_HandleTypeDef *htim, HAL_TIM_CallbackIDTypeDef CallbackID);
2547 #endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
2548
2549 /**
2550 * @}
2551 */

```

输入捕获中断回调函数

```

HAL_TIM_IC_Start_IT(&htim17, TIM_CHANNEL_1); // input capture interrupt enable

```

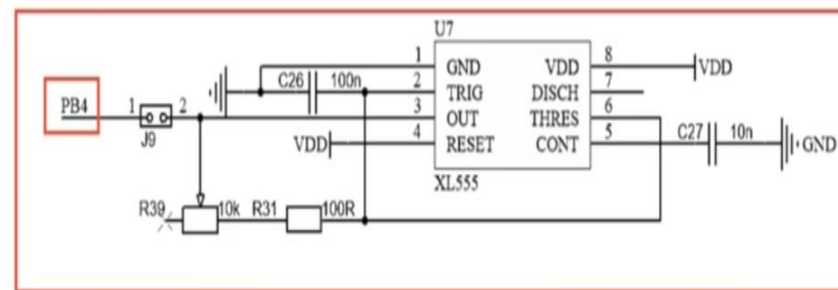
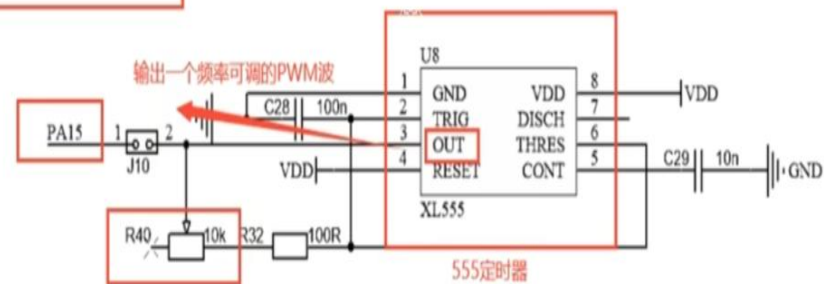
输入捕获中断使能函数



让PA1引脚输出PWM波

PA7

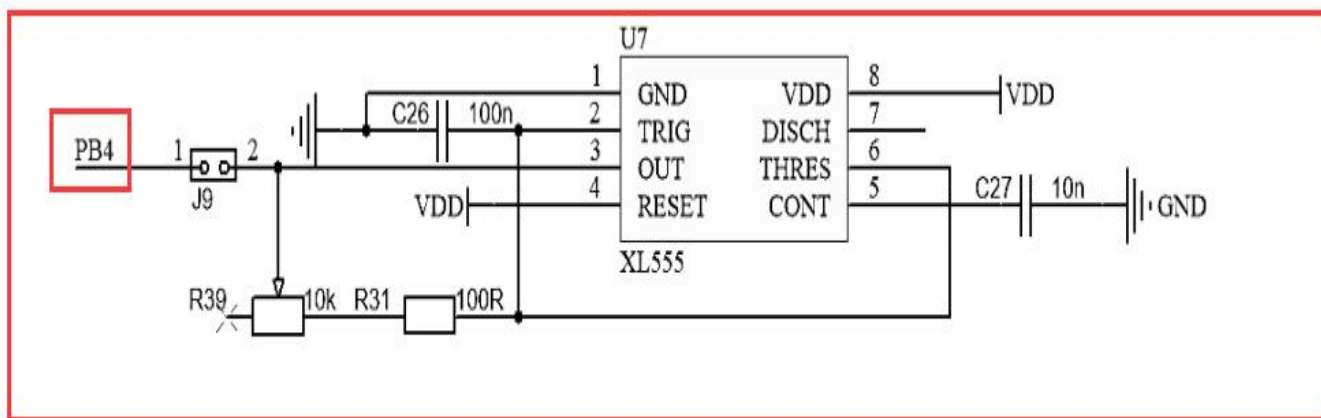
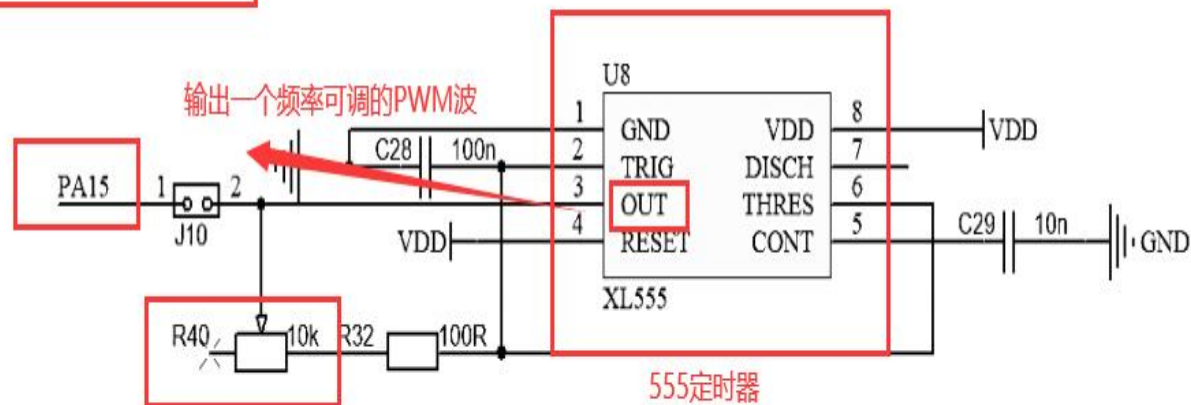
7.2 信号发生器



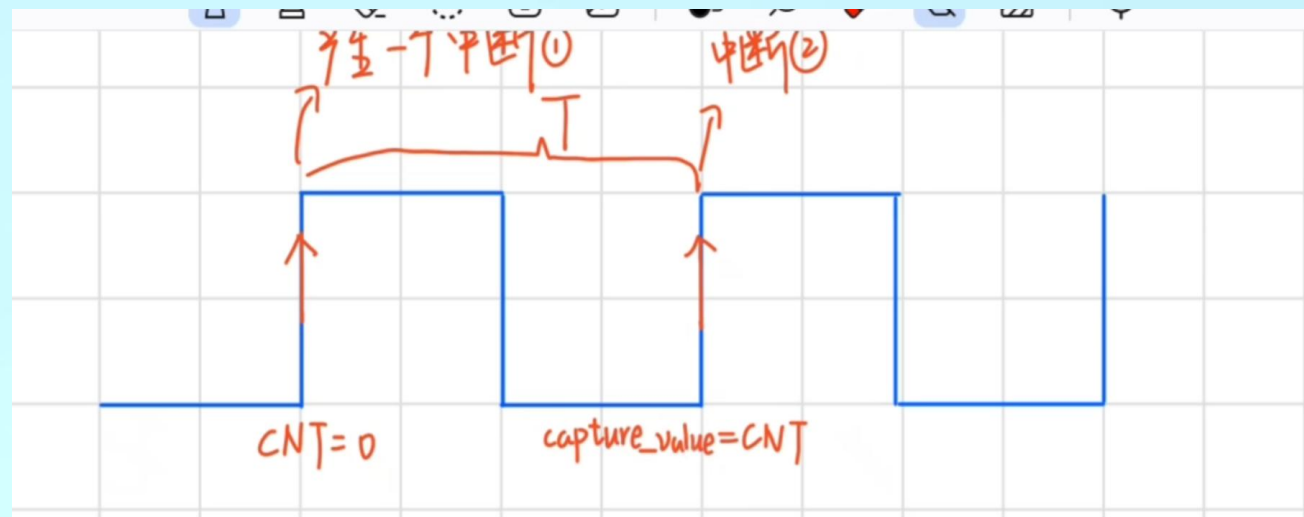
让PA1引脚输出PWM波

PA7

7.2 信号发生器



输入捕获频率 $f = \frac{f_{sys}}{(PSC + 1) * capture_value}$

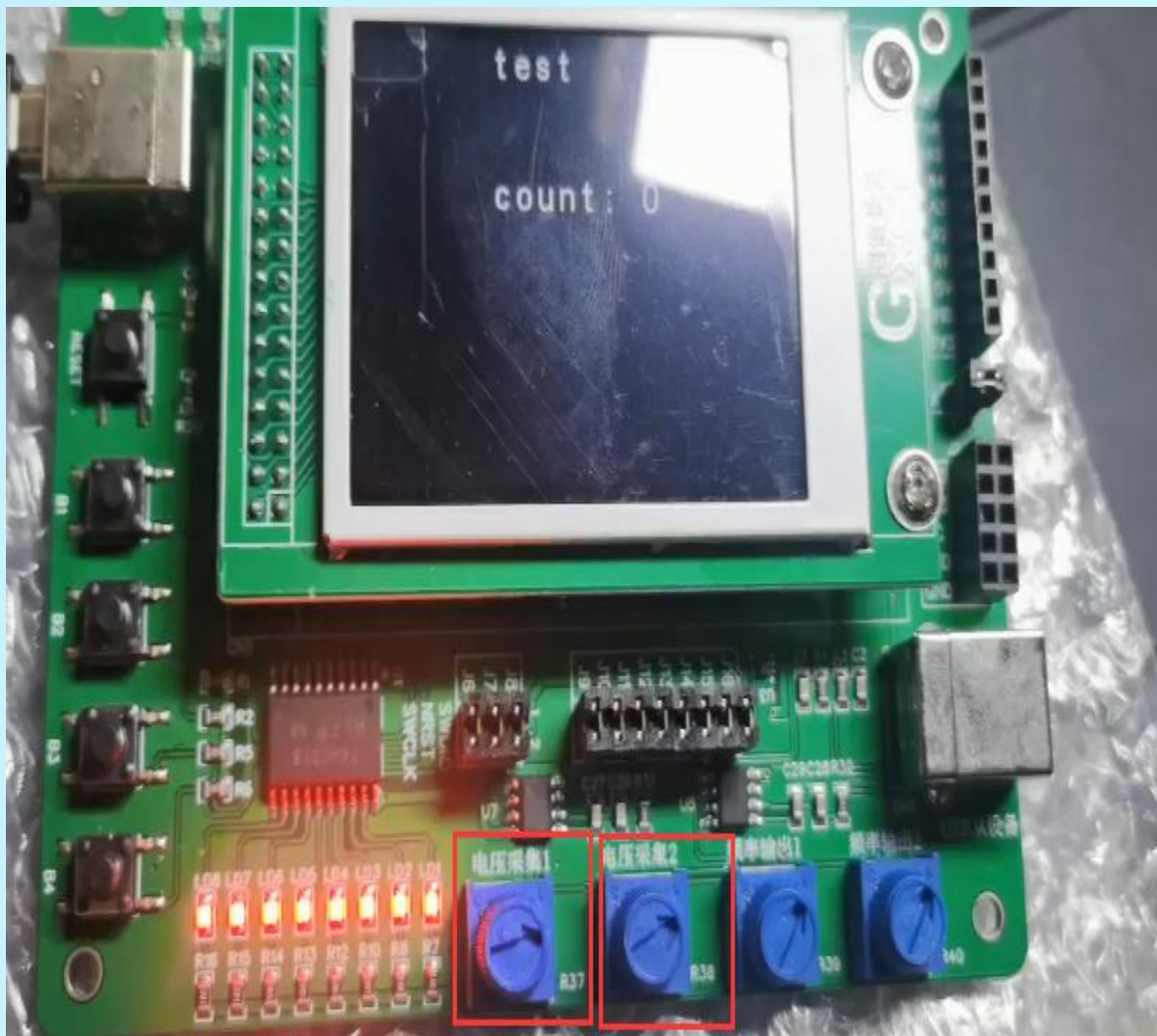


Timing diagram showing two pulses. The first pulse is labeled "CNT=0" and the second is labeled "capture_value=CNT".

$$t_0 = \frac{1}{\frac{f_{sys}}{PSC+1}} = \frac{PSC+1}{f_{sys}}$$

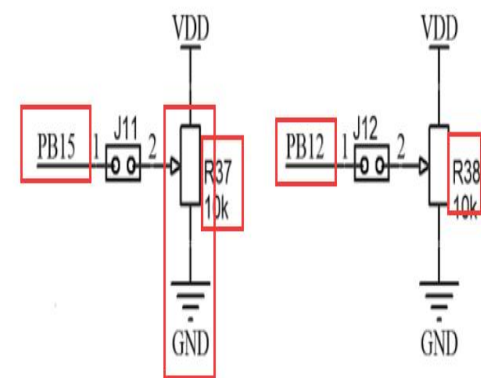
$$T = t_0 \cdot capture_value$$

★ $f = \frac{1}{T} = \frac{f_{sys}}{(PSC+1) \cdot capture_value}$

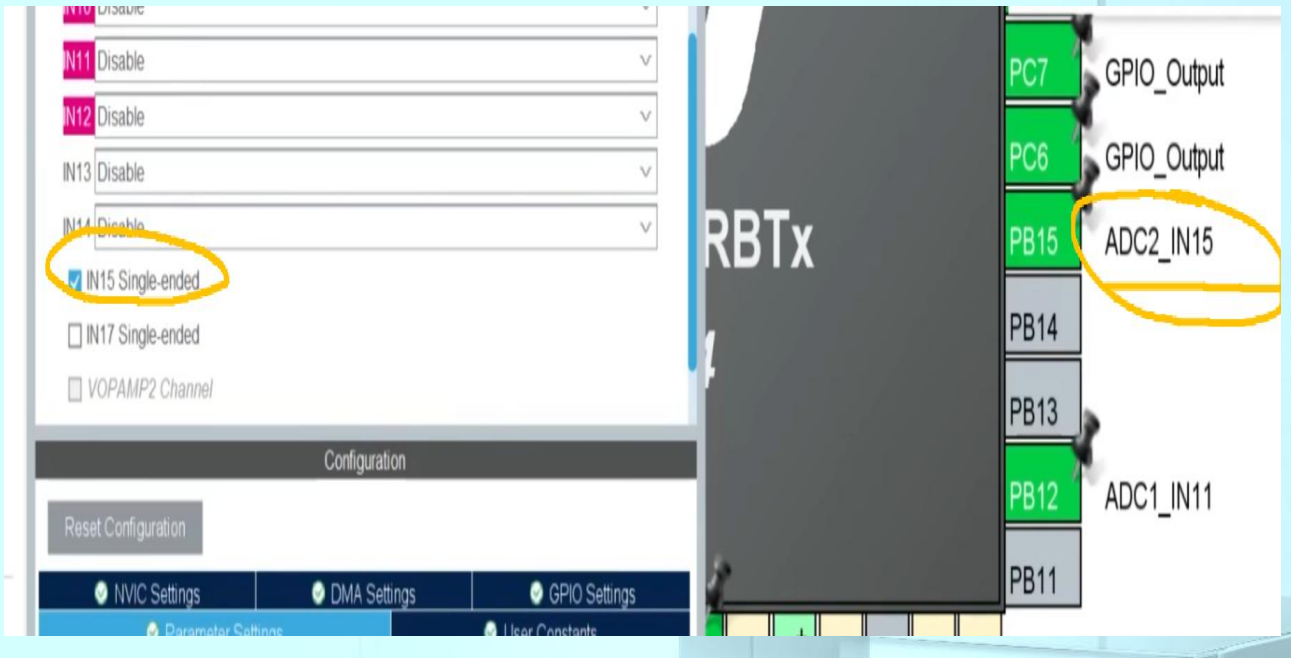
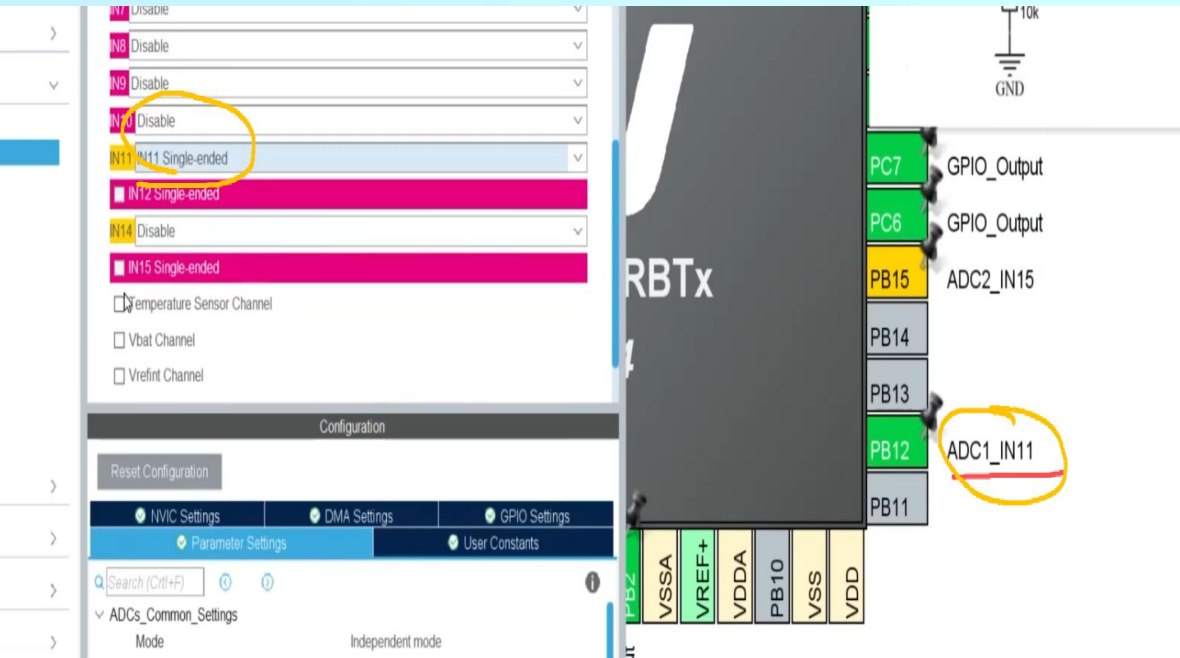
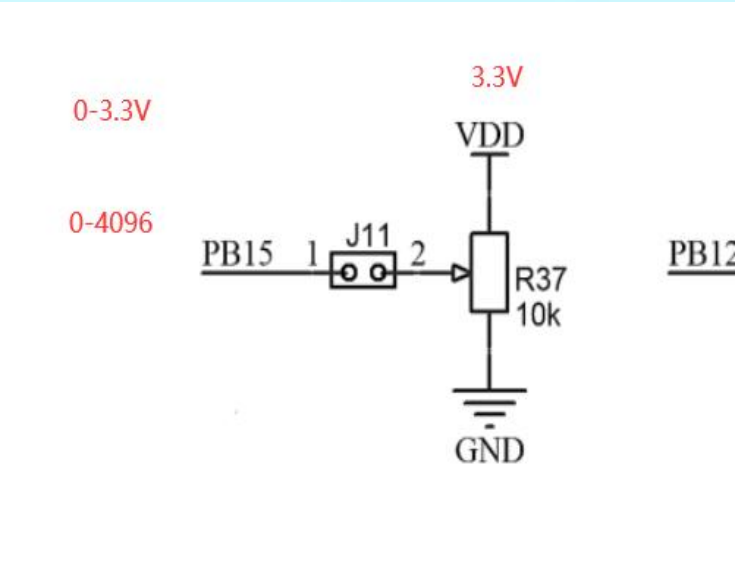
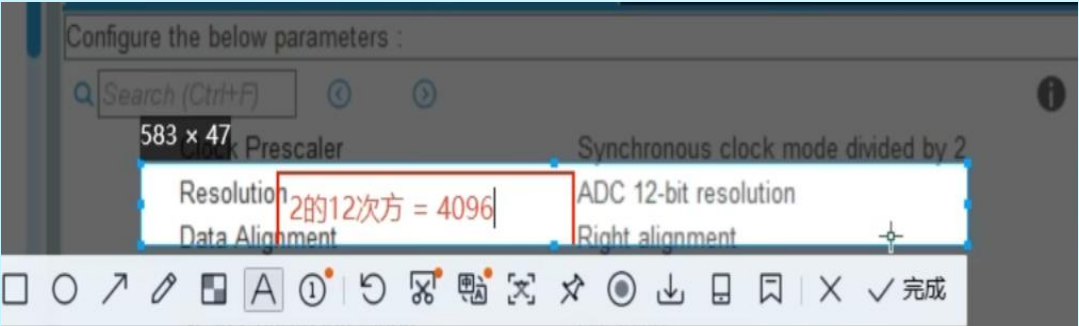


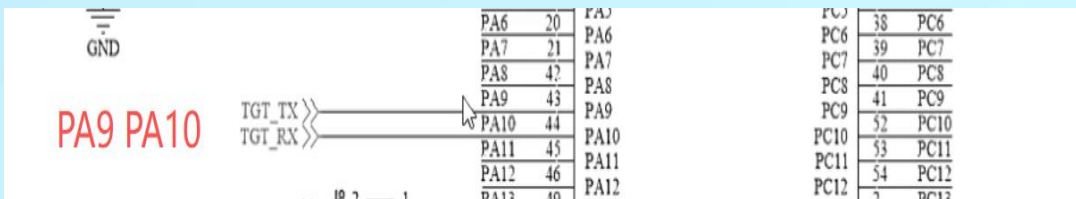
7.3 模拟输出

ADC输入模式

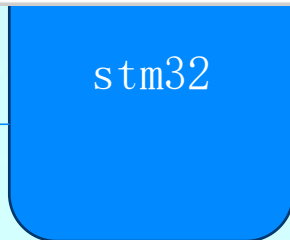


电压测量 $V = adc_value \frac{3.3}{4096}$





单个字节传输



```

1600 HAL_StatusTypeDef HAL_UART_Abort(UART_HandleTypeDef *huart);
1601 HAL_StatusTypeDef HAL_UART_AbortTransmit(UART_HandleTypeDef *huart);
1602 HAL_StatusTypeDef HAL_UART_AbortReceive(UART_HandleTypeDef *huart);
1603 HAL_StatusTypeDef HAL_UART_Abort_IT(UART_HandleTypeDef *huart);
1604 HAL_StatusTypeDef HAL_UART_AbortTransmit_IT(UART_HandleTypeDef *huart);
1605 HAL_StatusTypeDef HAL_UART_AbortReceive_IT(UART_HandleTypeDef *huart);
1606
1607 void HAL_UART_IRQHandler(UART_HandleTypeDef *huart);
1608 void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart);
1609 void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart);
1610 void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart);
1611 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
1612 void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart);
1613 void HAL_UART_AbortCpltCallback(UART_HandleTypeDef *huart);
1614 void HAL_UART_AbortTransmitCpltCallback(UART_HandleTypeDef *huart);
1615 void HAL_UART_AbortReceiveCpltCallback(UART_HandleTypeDef *huart);
1616
1617 /**
1618  * @}
1619  */
1620
1621 /** @addtogroup UART_Exported_Functions_Group3 Peripheral Control functions
1622  * @{
1623  */

```

串口接收中断回调函数

```

1588
1589 /* IO operation functions *****/
1590 HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
1591 HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
1592 HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1593 HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1594 HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1595 HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1596 HAL_StatusTypeDef HAL_UART_DMAMPAUSE(UART_HandleTypeDef *huart);
1597 HAL_StatusTypeDef HAL_UART_DMARESUME(UART_HandleTypeDef *huart);
1598 HAL_StatusTypeDef HAL_UART_DMASTOP(UART_HandleTypeDef *huart);
1599 /* Transfer Abort functions */
1600 HAL_StatusTypeDef HAL_UART_Abort(UART_HandleTypeDef *huart);
1601 HAL_StatusTypeDef HAL_UART_AbortTransmit(UART_HandleTypeDef *huart);
1602 HAL_StatusTypeDef HAL_UART_AbortReceive(UART_HandleTypeDef *huart);
1603 HAL_StatusTypeDef HAL_UART_Abort_IT(UART_HandleTypeDef *huart);
1604 HAL_StatusTypeDef HAL_UART_AbortTransmit_IT(UART_HandleTypeDef *huart);
1605 HAL_StatusTypeDef HAL_UART_AbortReceive_IT(UART_HandleTypeDef *huart);
1606
1607 void HAL_UART_IRQHandler(UART_HandleTypeDef *huart);
1608 void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart);
1609 void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart);

```

串口接收数据的函数

```

1573
1574 /* Callbacks Register/UnRegister functions *****/
1575 #if (USE_HAL_UART_REGISTER_CALLBACKS == 1)
1576 HAL_StatusTypeDef HAL_UART_RegisterCallback(UART_HandleTypeDef *huart, HAL_UART_CallbackIDTypeDef CallbackID,
1577                                             pUART_CallbackTypeDef pCallback);
1578 HAL_StatusTypeDef HAL_UART_UnRegisterCallback(UART_HandleTypeDef *huart, HAL_UART_CallbackIDTypeDef CallbackID);
1579 #endif /* USE_HAL_UART_REGISTER_CALLBACKS */
1580
1581 /**
1582  * @}
1583  */
1584
1585 /** @addtogroup UART_Exported_Functions_Group2 IO operation functions
1586  * @{
1587  */
1588
1589 /* IO operation functions *****/
1590 HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
1591 HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
1592 HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1593 HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1594 HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1595 HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size);
1596 HAL_StatusTypeDef HAL_UART_DMAMPAUSE(UART_HandleTypeDef *huart);
1597 HAL_StatusTypeDef HAL_UART_DMARESUME(UART_HandleTypeDef *huart);
1598 HAL_StatusTypeDef HAL_UART_DMASTOP(UART_HandleTypeDef *huart);
1599 /* Transfer Abort functions */

```

串口接收：每次进入中断 只能接收一个字节的的数据

考虑错误情况，1. 每次接收一个字节就要判断该字节是否符合要求

是否符合要求

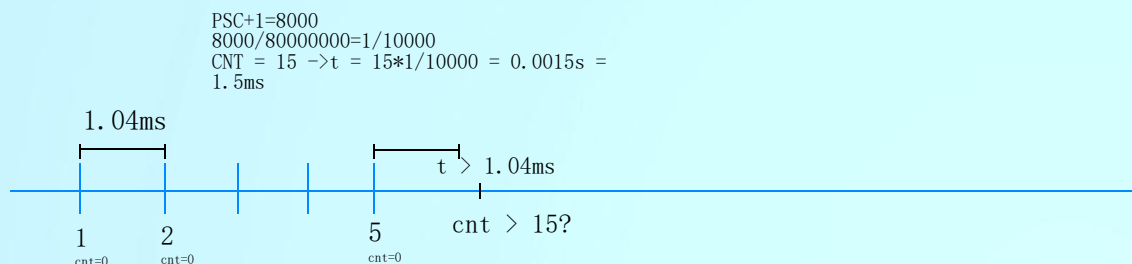
2. 接收完所有字符，判断字符串是

解决方法：利用定时器，处理串口接收

串口波特率 = 9600 bit/s 就是1s可以传输9600bit

串口传输一次数据 包含起始位，数据位，结束位，一共10bit

$$10 * 1/9600 = 0.00104s = 1.04ms$$



eeeprom

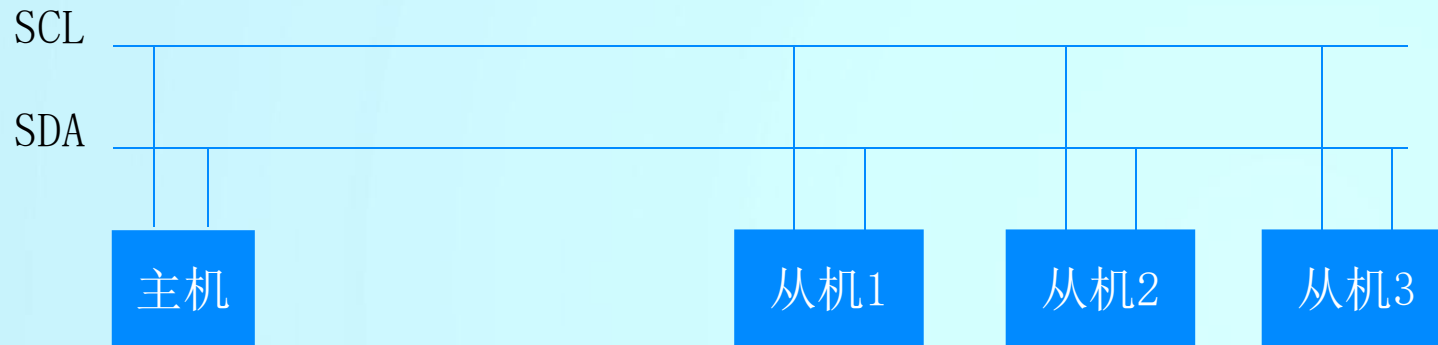
非易失性存储器（掉电后数据不丢失）

片内地址

记住两点：

1. 主机给从机发信息，从机接到信息后要给主机回应
2. 从机给主机发信息，主机接到信息后也要给从机回应

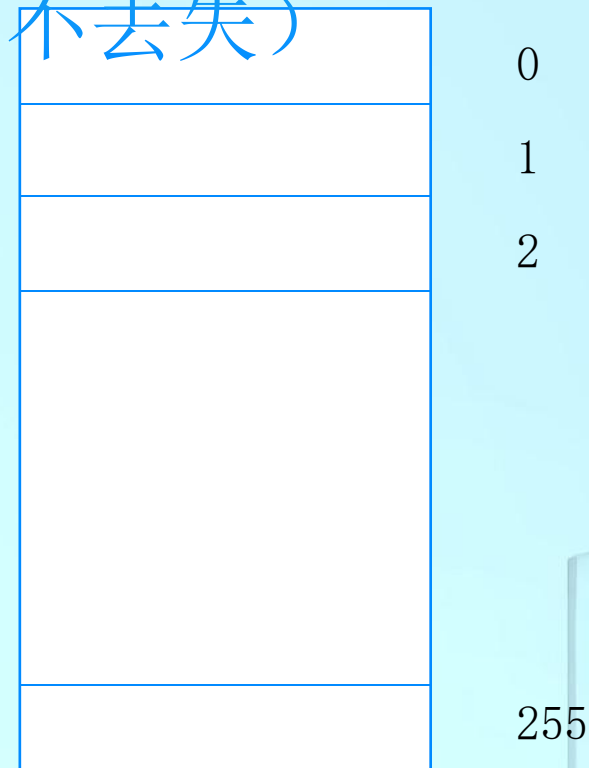
IIC通信协议



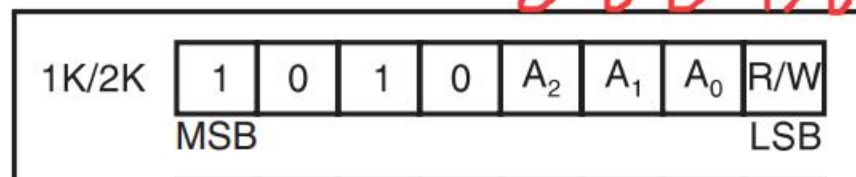
STM32

AT24C02

地址：a0

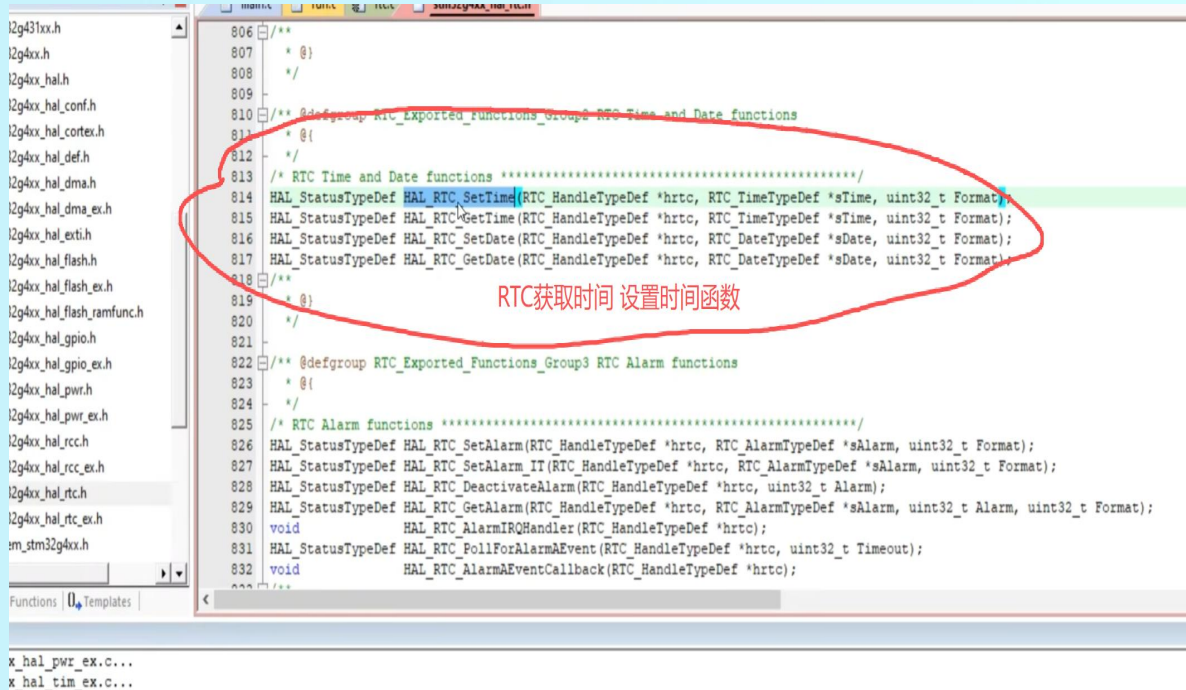


AT24C02



RTC 实时时钟

- 主要功能实现：
1. 设置时间和日期
 2. 读取时间和日期
 3. 设置一个闹钟



```
gpio.c
7 HAL_GPIO_WritePin(GPIOC, GPIO_PIN_8 << (led - 1), GPIO_PIN_RESET);
8 else
9 HAL_GPIO_WritePin(GPIOC, GPIO_PIN_8 << (led - 1), GPIO_PIN_SET);
10 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_2, GPIO_PIN_RESET);
11 }
12
13
14 char text[20];
15 RTC_TimeTypeDef sTime = {0};
16 RTC_DateTypeDef sDate = {0};
17
18
19 HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
20 HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);
21 sprintf(text, " test ");
22 LCD_DisplayStringLine(Line0, (uint8_t *)text);
23 sprintf(text, " %2d:%2d:%2d ", sTime.Hours, sTime.Minutes);
24 LCD_DisplayStringLine(Line0, (uint8_t *)text);
25 sprintf(text, " test ");
26 LCD_DisplayStringLine(Line0, (uint8_t *)text);
27 }
```

注意：即使你不需要获取日期，也要将获取日期的函数添上，不然的话时间就不会流动。

```
818 /**
819  * @}
820  */
821
822 /** @defgroup RTC_Exported_Functions_Group3 RTC Alarm functions
823  * @{
824  */
825 /* RTC Alarm functions */
826 HAL_StatusTypeDef HAL_RTC_SetAlarm(RTC_HandleTypeDef *hrtc, RTC_AlarmTypeDef *sAlarm, uint32_t Format);
827 HAL_StatusTypeDef HAL_RTC_SetAlarm_IT(RTC_HandleTypeDef *hrtc, RTC_AlarmTypeDef *sAlarm, uint32_t Format);
828 HAL_StatusTypeDef HAL_RTC_DeactivateAlarm(RTC_HandleTypeDef *hrtc, uint32_t Alarm);
829 HAL_StatusTypeDef HAL_RTC_GetAlarm(RTC_HandleTypeDef *hrtc, RTC_AlarmTypeDef *sAlarm, uint32_t Alarm, uint32_t Format);
830 void HAL_RTC_AlarmIRQHandler(RTC_HandleTypeDef *hrtc);
831 HAL_StatusTypeDef HAL_RTC_PollForAlarmEvent(RTC_HandleTypeDef *hrtc, uint32_t Timeout);
832 void HAL_RTC_AlarmEventCallback(RTC_HandleTypeDef *hrtc);
833 /**
834  * @}
835  */
836
837 /** @defgroup RTC_Exported_Functions_Group4 Peripheral Control functions
838  * @{
839  */
840 /* Peripheral Control functions */
841 HAL_StatusTypeDef HAL_RTC_WaitForSynchro(RTC_HandleTypeDef *hrtc);
```

闹钟中断回调函数